Snowflake + GitHub Actions Authentication — Developer Cheatsheet

Connect GitHub Actions to Snowflake securely: OIDC (secretless, recommended) or Key Pair/PAT as fallback for older tooling.

> [!IMPORTANT] Community cheatsheet — not official Snowflake documentation. For the authoritative reference, see Snowflake CLI GitHub Action docs.

Table of Contents

Choose Your Method

Method	GitHub Secrets needed	Credential rotation	Snow CLI
OIDC (recommended)	`SNOWFLAKE_ACCOUNT` only	None — tokens short-lived per run	≥ 3.11
Key pair	Account + user + private key	Yes — rotate private key	Any
PAT / Password	Account + user + password/PAT	Yes — manual rotation	Any

Check your installed version:

```
# run in terminal
snow --version   # must be ≥ 3.11 for OIDC; upgrade: pip install -
U snowflake-cli
```

> [!TIP] OIDC is the only method that stores zero long-lived secrets in GitHub. Use it whenever Snow CLI ≥ 3.11 is available.

OIDC Setup: Snowflake Side

Run as `ACCOUNTADMIN`. Creates a service user that trusts GitHub's OIDC tokens directly — no password, no key pair stored anywhere.

```
USE ROLE ACCOUNTADMIN;

-- 1. Least-privilege role for the workflow
CREATE ROLE IF NOT EXISTS gh_actions_role
  COMMENT = 'Least-privilege role for GitHub Actions via OIDC';

GRANT USAGE ON WAREHOUSE my_wh TO ROLE gh_actions_role;
-- Add further grants only as required by the workflow

-- 2. Service user mapped to GitHub OIDC
CREATE USER IF NOT EXISTS gh_actions_svc_user
  TYPE    = SERVICE
  WORKLOAD_IDENTITY = (
    TYPE    = OIDC
    ISSUER  = 'https://token.actions.githubusercontent.com'
    SUBJECT = 'repo:<owner>/<repo>:environment:<env-name>'
  )
  DEFAULT_ROLE      = gh_actions_role
  DEFAULT_WAREHOUSE = my_wh
  COMMENT           = 'GitHub Actions OIDC service user';

GRANT ROLE gh_actions_role TO USER gh_actions_svc_user;
```

Verify the user and workload identity were created correctly:

```
DESCRIBE USER gh_actions_svc_user;
SHOW PARAMETERS LIKE 'DEFAULT_ROLE' FOR USER gh_actions_svc_user;
```

Replace `<owner>/<repo>:environment:<env-name>` with your values — see OIDC Subject Patterns.

OIDC Setup: GitHub Actions

Two requirements: `permissions: id-token: write` on the job and `use-oidc: true` on the action. Only one GitHub Secret is needed: `SNOWFLAKE_ACCOUNT`.

```yaml
name: Deploy to Snowflake
on:
  push:
    branches: [main]

permissions:
  id-token: write   # required for OIDC token issuance
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: prod         # must match the SUBJECT in CREATE
        USER
    steps:
      - uses: actions/checkout@v4
        with:
          persist-credentials: false

      - uses: snowflakedb/snowflake-cli-action@v2
        with:
          use-oidc: true
          cli-version: "3.16"

      - name: Test connection
        env:
          SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
        run: snow connection test -x

      - name: Run governance SQL
        env:
          SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
        run: snow sql -x -f governance/policy.sql
```

`-x` (`--temporary-connection`) builds the connection from env vars — no `config.toml` required on the runner.

Set the one required GitHub Secret:

```
# run in terminal
gh secret set SNOWFLAKE_ACCOUNT --body "orgname-accountname"
```

OIDC Subject Patterns

The `SUBJECT` in `CREATE USER` must match the claim GitHub emits for the job exactly.

Subject format	Matches	Job requirement
`repo:<owner>/<repo>:environment:<name>`	Job targets a named environment	`environment: <name>` set on the job **(recommended)**
`repo:<owner>/<repo>:ref:refs/heads/<branch>`	Push to the specified branch	`on: push`, no `environment:` on job
`repo:<owner>/<repo>:pull_request`	Any pull request event	`on: pull_request`, no `environment:`

> [!NOTE] When a job sets `environment:`, GitHub always uses the environment-based subject regardless of trigger. Use `environment:` subjects for tighter scoping — one OIDC user per environment (e.g. `dev`, `prod`).

Network Policy

Only needed if your Snowflake account restricts inbound IPs. Use the Snowflake-managed rule — no manual CIDR list maintenance required.

Check whether the service user already has a network policy:

```
SHOW PARAMETERS LIKE 'NETWORK_POLICY' FOR USER
        gh_actions_svc_user;
```

Create and apply a new policy with the managed GitHub Actions rule:

```
CREATE NETWORK POLICY gh_actions_policy
  ALLOWED_NETWORK_RULE_LIST =
        ('SNOWFLAKE.NETWORK_SECURITY.GITHUBACTIONS_GLOBAL');

ALTER USER gh_actions_svc_user
  SET NETWORK_POLICY = gh_actions_policy;
```

Add to an existing policy instead of replacing it:

```
ALTER NETWORK POLICY existing_policy
  ADD ALLOWED_NETWORK_RULE_LIST =
        ('SNOWFLAKE.NETWORK_SECURITY.GITHUBACTIONS_GLOBAL');
```

> [!NOTE] If you see `Incoming request with IP is not allowed to access Snowflake` — add the network policy. If the account has no IP restrictions, skip this section.

Key Pair Auth

Use when Snow CLI < 3.11 or OIDC is not available. GitHub Secrets needed: `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_PRIVATE_KEY_RAW`.

Generate a key pair:

```
# run in terminal
openssl genrsa 2048 | openssl pkcs8 -topk8 -nocrypt -out
private_key.p8
openssl rsa -in private_key.p8 -pubout -out public_key.pub
```

Register the public key on the Snowflake user:

```
ALTER USER my_user SET RSA_PUBLIC_KEY =
        '<public_key.pub contents, no header/footer>';
```

Store all three required GitHub Secrets:

```
# run in terminal
gh secret set SNOWFLAKE_ACCOUNT        --body "orgname-
accountname"
gh secret set SNOWFLAKE_USER           --body "my_user"
gh secret set SNOWFLAKE_PRIVATE_KEY_RAW < private_key.p8
```

Temporary connection (no `config.toml` — recommended for CI):

```
- uses: snowflakedb/snowflake-cli-action@v2
  with:
    cli-version: "3.16"

- name: Deploy
  env:
    SNOWFLAKE_AUTHENTICATOR:    SNOWFLAKE_JWT
    SNOWFLAKE_ACCOUNT:          ${{ secrets.SNOWFLAKE_ACCOUNT }}
    SNOWFLAKE_USER:             ${{ secrets.SNOWFLAKE_USER }}
    SNOWFLAKE_PRIVATE_KEY_RAW:  $
        {{ secrets.SNOWFLAKE_PRIVATE_KEY_RAW }}
  run: snow connection test -x
```

Named connection (commit a minimal `config.toml`, override via env vars):

```
# config.toml — commit this, contains no secrets
default_connection_name = "ci"
```

```
[connections.ci]
```

```
- uses: snowflakedb/snowflake-cli-action@v2
  with:
    cli-version: "3.16"
    default-config-file-path: "config.toml"

- name: Deploy
  env:
    SNOWFLAKE_CONNECTIONS_CI_AUTHENTICATOR:   SNOWFLAKE_JWT
    SNOWFLAKE_CONNECTIONS_CI_ACCOUNT:         $
        {{ secrets.SNOWFLAKE_ACCOUNT }}
    SNOWFLAKE_CONNECTIONS_CI_USER:            $
        {{ secrets.SNOWFLAKE_USER }}
    SNOWFLAKE_CONNECTIONS_CI_PRIVATE_KEY_RAW: $
        {{ secrets.SNOWFLAKE_PRIVATE_KEY_RAW }}
  run: snow connection test
```

PAT / Password (Legacy)

Use only for legacy tooling that cannot use OIDC or key pairs. GitHub Secrets needed: `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_PASSWORD` (or PAT).

Generate a PAT in Snowsight (Profile → Programmatic Access Tokens → Generate) or via SQL:

```
ALTER USER my_user ADD PROGRAMMATIC ACCESS TOKEN my_ci_token
  ROLE_RESTRICTION = my_ci_role;
```

GitHub Actions workflow:

```
- uses: snowflakedb/snowflake-cli-action@v2
  with:
    cli-version: "3.16"

- name: Run SQL
  env:
    SNOWFLAKE_ACCOUNT:  ${{ secrets.SNOWFLAKE_ACCOUNT }}
    SNOWFLAKE_USER:     ${{ secrets.SNOWFLAKE_USER }}
    SNOWFLAKE_PASSWORD: ${{ secrets.SNOWFLAKE_PASSWORD }}
  run: snow connection test -x
```

Remove a PAT when no longer needed:

```
ALTER USER my_user REMOVE PROGRAMMATIC ACCESS TOKEN my_ci_token;
```

Env Vars Reference

Variables set automatically by `snowflake-cli-action@v2` when `use-oidc: true`. You only need to provide `SNOWFLAKE_ACCOUNT` — everything else is injected by the action.

Variable	Value set by action
`SNOWFLAKE_AUTHENTICATOR`	`WORKLOAD_IDENTITY`
`SNOWFLAKE_WORKLOAD_IDENTITY_PROVIDER`	`OIDC`
`SNOWFLAKE_AUDIENCE`	`snowflakecomputing.com`
`SNOWFLAKE_TOKEN`	GitHub OIDC token (short-lived, per run)

Override the token variable for named connections:

```yaml
- uses: snowflakedb/snowflake-cli-action@v2
  with:
    use-oidc: true
    oidc-token-name: SNOWFLAKE_CONNECTIONS_PROD_TOKEN
```

Use `oidc-token-name: SNOWFLAKE_CONNECTIONS_<NAME>_TOKEN` where `<NAME>` is the uppercased connection name from `config.toml` (e.g. `[connections.prod]` → `PROD`).

Cleanup

Remove OIDC objects when decommissioning a workflow:

```sql
USE ROLE ACCOUNTADMIN;

DROP USER IF EXISTS gh_actions_svc_user;
DROP ROLE IF EXISTS gh_actions_role;
DROP NETWORK POLICY IF EXISTS gh_actions_policy;
```

Remove GitHub Secrets via CLI:

```
# run in terminal
gh secret delete SNOWFLAKE_ACCOUNT
gh secret delete SNOWFLAKE_USER             # key pair / PAT only
gh secret delete SNOWFLAKE_PRIVATE_KEY_RAW  # key pair only
gh secret delete SNOWFLAKE_PASSWORD         # PAT / password only
```

Tips & Gotchas

- **Subject must match exactly.** A mismatch between `SUBJECT` in `CREATE USER` and the claim GitHub emits silently fails auth with

a generic error. Test with `snow connection test -x` in a throwaway workflow run before building the full pipeline.

- **`environment:` overrides all other subject formats.** Once a job sets `environment:`, GitHub always uses the environment claim regardless of trigger (push, PR, schedule). Plan one OIDC service user per environment (`dev`, `staging`, `prod`), not per branch.
- **`-x` is required in CI.** Without `--temporary-connection`, Snow CLI looks for `~/.snowflake/config.toml` which does not exist on a fresh GitHub Actions runner.
- **Network policy is evaluated before OIDC token validation.** An IP-blocked runner shows `IP not allowed` — not an auth error. Add `SNOWFLAKE.NETWORK_SECURITY.GITHUBACTIONS_GLOBAL` first, then debug the OIDC subject.
- **PATs expire silently.** A workflow using an expired PAT fails without a clear error until someone investigates. Set a calendar reminder before `DEFAULT_EXPIRY_IN_DAYS`. OIDC tokens are always fresh — one more reason to prefer OIDC.

References

- Snowflake CLI GitHub Action
- Integrating CI/CD with Snowflake CLI
- snow auth oidc commands
- Snowflake Programmatic Access Tokens
- Snowflake Network Rules